

Computational Engineering

Google Page Rank

Die Benutzung des Internets ist ohne eine funktionale Suchmaschine praktisch kaum denkbar. Eine Suchmaschine bildet dabei durch einen internen Algorithmus eine **Reihenfolge** der **Wichtigkeit** einzelner Seiten im Internet, die zum Beispiel einen bestimmten Suchbegriff enthalten.

Es stellt sich die Frage wie man eine solche Reihenfolge effizient berechnen kann. Das Ziel dieser Veranstaltungseinheit ist es, die Idee der Google-Gründer S. Brin und L. Page nachzuvollziehen und das entsprechende Modell zu verstehen und nachzuprogrammieren. Die Suchmaschine von Google arbeitet dabei mittels zweier Instanzen:

- **Index:** Im Index sind alle zugänglichen Seiten mit ihren Inhalten erfasst und die relevanten Schlagwörter in einer Datenbank abgelegt. Nach Eingabe einer Suchanfrage werden aus dem Index alle Seiten heraus gesucht, die die gesuchten Kriterien erfüllen.
- **Ranking:** Die Trefferliste wird nun an Hand der **Wichtigkeit/Relevanz** nach fest gelegten Regeln **sortiert**.

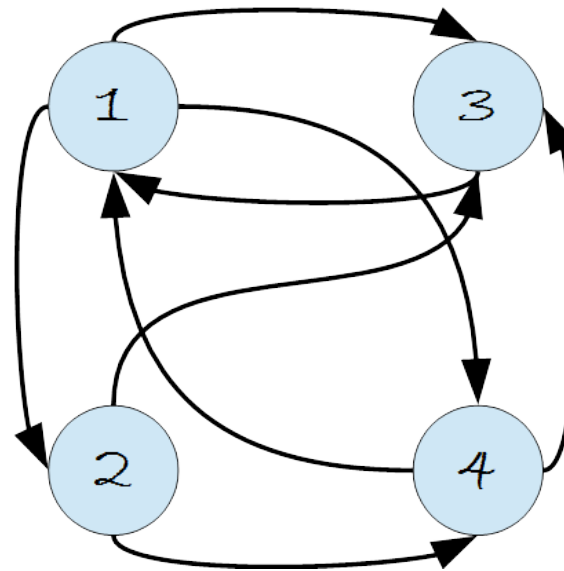
Der tatsächlich eingesetzte Algorithmus ist sehr kompliziert (und nicht im Detail öffentlich bekannt) und berücksichtigt mehrere, sowohl anfrageabhängige (inhaltsbasierte), als auch anfrageunabhängige Faktoren. Letzteres sind zum Beispiel die Verlinkungsstruktur mit anderen Seiten, d.h. insbesondere die Häufigkeit der Verweise von und zu der betreffenden Seite.

Wir stellen uns deshalb nur dem Problem eine **sinnvolle** (und effiziente) Methode zu definieren, um eine anfrageunabhängige Reihenfolge der Wichtigkeit aller Webseiten zu berechnen.

Die Begriffe **Wichtigkeit**, **Relevanz** oder **Qualität** einer Webseite sind nicht eindeutig definiert und wir müssen eine mathematisch so präzise Definition suchen, dass sich daraus eine **Berechnungsvorschrift** ableiten lässt. Der Google-Ansatz hat dabei seine Wurzeln in der Bibliographie. Hier wird die Wichtigkeit eines wissenschaftlichen Aufsatzes zumeist daran gemessen, wie viele andere Aufsätze diesen **zitieren**, d.h. auf diesen **verweisen**.

Im Internet verweisen Seiten auf andere Seiten durch **Verlinkung**. Im einfachsten Modell wird ein solcher Link als **Empfehlung** der jeweiligen Seite angesehen. Die reine Anzahl von Verweisen als Qualitätsmerkmal ist aber zu eindimensional. Der Google-Ansatz berücksichtigt daher Verweise von wichtigen Seiten stärker.

Zur Veranschaulichung betrachten wir ein Mini-Internet aus nur vier Seiten, dessen Link-Struktur in der folgenden Graphik abgebildet ist.



Wenn man nur die **Zahl der eingehenden Links** als Merkmal der Wichtigkeit heranzieht, ergibt sich die Reihenfolge $S_3, (S_1, S_4), S_2$, wobei Seite 1 und Seite 4 einen identischen Rang bekleiden.

Welche Reihenfolge ergibt sich wohl, wenn die (noch nicht bekannte) Wichtigkeit einer verweisenden Seite mit in Betracht gezogen wird?

Will man die Wichtigkeit einer **empfehlenden** Seite mit einfließen lassen, damit ein eingehender Link umso wertvoller wird, je wichtiger die empfehlende Seite ist, lässt sich dies in folgendem einfachen Modell realisieren.

- Das betrachtete Internet habe n Seiten, die i -te Seite werde mit S_i bezeichnet.
- Die Wichtigkeit der Seite S_i ist durch den **Page Rank** r_i gekennzeichnet. Dabei ist die Seite S_i wichtiger als die Seite S_j , wenn $r_i > r_j$.
- Mit $|S_i|$ sei die Anzahl der von der Seite S_i **ausgehenden** Links bezeichnet.
- Wenn die Seite S_i einen Link auf die Seite S_j enthält, bekommt die Seite S_j dann den $1/|S_i|$ -ten Teil des Page Ranks der Seite S_i übertragen. Wenn eine Seite weniger Links enthält, ist dadurch ein Link von ihr wertvoller, als wenn sie viele Links enthält. Das wertet so genannte Linkfarmen ab.
- Die Menge aller Seiten, die einen Link auf die Seite S_i enthalten bezeichnen wir mit B_i .
- Die unbekannten Page Ranks sind dann Lösung des **linearen Gleichungssystems**

$$r_i = \sum_{S_j \in B_i} \frac{r_j}{|S_j|}, \quad i = 1, \dots, n \quad (1)$$

Mittels des Page Rank Vektors

$$r = (r_1, r_2, \dots, r_n)^t$$

und der **Hyperlink-Matrix** $H = (h_{ij}) \in \mathbb{R}^{n \times n}$ mit den Elementen

$$h_{ij} = \begin{cases} \frac{1}{|S_i|}, & \text{wenn } S_i \in B_j \\ 0 & \text{sonst} \end{cases}$$

lässt sich Gl. (1) kompakt schreiben als

$$H^t \cdot r = r \tag{2}$$

Die gesuchten Page Ranks sind also die Koordinaten des Eigenvektors der transponierten Hyperlink-Matrix zum Eigenwert 1.

Das Mini-Internet mit 4 Seiten von Folie 4 hat die Hyperlink-Matrix

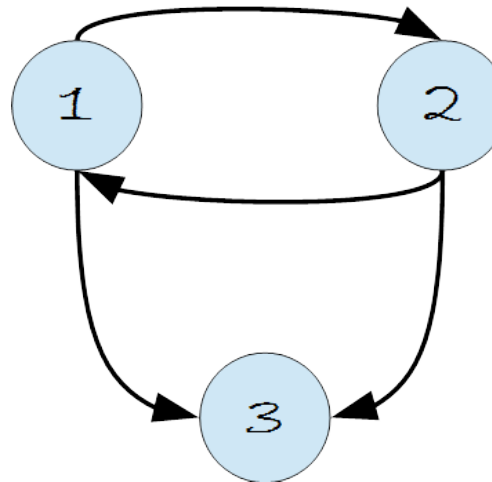
$$H = \begin{pmatrix} 0 & 1/3 & 1/3 & 1/3 \\ 0 & 0 & 1/2 & 1/2 \\ 1 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 \end{pmatrix}$$

und die (bis auf skalare Vielfache) eindeutige Lösung der Gl. (2) ist

$$r^t = (0.3871, 0.1290, 0.2903, 0.1935)$$

Alle Einträge des Page Rank Vektors haben das gleiche Vorzeichen, was bedeutet, dass alle positiv gewählt werden können. Das ist nicht ohne weiteres klar aber notwendig dafür, dass die Einträge als Wichtigkeiten interpretiert werden können. Die Reihenfolge der Wichtigkeiten die sich ergibt, ist hier also S_1, S_3, S_4, S_2

Das vorliegende Modell ist nicht allgemein nutzbar. Zum Beispiel enthält die Hyperlink-Matrix des zweiten Beispiels eines Mini-Internets aus nur drei Seiten



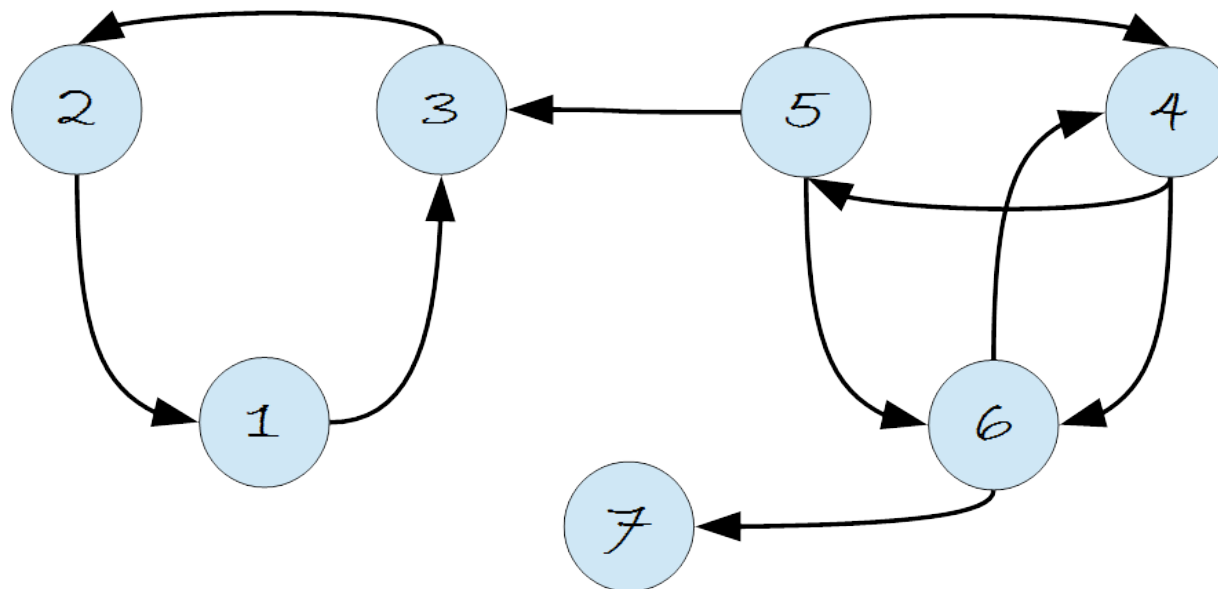
eine Nullzeile, da die Seite S_3 keine ausgehenden Links hat. Deshalb hat die Eigenwertgleichung (2) in diesem Fall keine eindeutige Lösung. Man bezeichnet in diesem Zusammenhang eine Seite ohne ausgehende Links als "hängenden Knoten".

Wir wollen ein Modell, welches für jedes Internet mit n Seiten und bekannter Link-Struktur einen (bis auf skalare Vielfache) eindeutigen Page Rank Vektor mit nicht negativen Einträgen liefert. Die Idee der Google-Gründer war es, einen **Zufalls-Surfer** einzuführen, der nach einfachen Regeln von Seite zu Seite durch das Internet springt. Der Page Rank ergibt sich dann aus der **durchschnittlichen Aufenthaltsdauer** des Zufallssurfers auf einer der Seiten des Internets.

Die einfachste Annahme über die Verhaltensregeln des Zufallssurfers sind dann die folgenden:

- Der Zufallssurfer ist gedächtnislos. Sein Verhalten hängt nur von der Seite ab auf der er sich gerade befindet.
- Jeder Iterationsschritt entspricht einer Zeiteinheit. Der Zufallssurfer springt in jeder Zeiteinheit von der Seite auf der er sich befindet auf eine andere Seite, in dem er
 - 1 zufällig gleichverteilt einem der Links auf der momentanen Seite folgt, falls Links vorhanden sind, oder
 - 2 zufällig gleichverteilt auf irgendeine Seite im Internet springt, falls keine Links auf der momentanen Seite vorhanden sind.

Das Modell des Zufallssurfers vermeidet das Problem mit den hängenden Knoten, kann aber so noch nicht eingesetzt werden. Als anschauliches Beispiel für ein Mini-Internet, in dem das vorliegende Modell offenbar zu einem nicht gewünschten Verhalten führt, sehen wir uns die folgende Graphik an



Gelangt der Zufalls-Surfer irgendwann auf eine der Seiten S_1 , S_2 oder S_3 , dann kommt er nie wieder aus dem geschlossenen Zyklus der drei Seiten heraus und im Grenzwert unendlicher Laufzeiten ergibt sich für die Wichtigkeiten der Seiten S_4 bis S_7 Null.

Um sicher zu stellen, dass tatsächlich jede Seite des Internets von jeder anderen Seite aus erreicht werden kann, modifizieren wir die Verhaltensregeln des Zufalls-Surfers.

- Der Zufallssurfer ist gedächtnislos. Sein Verhalten hängt nur von der Seite ab auf der er sich gerade befindet.
- Jeder Iterationsschritt entspricht einer Zeiteinheit. Der Zufallssurfer springt in jeder Zeiteinheit von der Seite auf der er sich befindet auf eine andere Seite, in dem er
 - **mit der Wahrscheinlichkeit** $\alpha \in (0, 1)$
 - 1 zufällig gleichverteilt einem der Links auf der momentanen Seite folgt, falls Links vorhanden sind, oder
 - 2 zufällig gleichverteilt auf irgendeine Seite im Internet springt, falls keine Links auf der momentanen Seite vorhanden sind.
 - und mit der Wahrscheinlichkeit $1 - \alpha$ zufällig gleichverteilt auf eine beliebige Seite im Internet springt.

Die Wahl des sogenannten Teleportationsparameters α ist nicht eindeutig fest gelegt. In der Praxis erweist sich ein Wert von $\alpha \approx 0.85$ oft als brauchbar.

In der Praxis formuliert man das Modell des Zufallssurfers nicht als Monte-Carlo-Simulation, da dies für ein Internet aus vielen Millionen von Seiten nicht mehr effizient zu handhaben wäre, sondern analytisch im Rahmen der Theorie der **Markov-Prozesse**. In dieser Formulierung erhält man den Page Rank Vektor dann wieder als Lösung eines (iterativen) Eigenwertproblems. Dieses ist dann abhängig von dem Parameter α .

Das Problem der hängenden Knoten lässt sich in der Hyperlinkmatrix einfach lösen. Wenn in der Matrix H eine Nullzeile vorkommt (das entspricht einer Seite ohne ausgehende Links), dann ersetzt man diese Nullzeile folgendermaßen:

$$(0, 0, \dots, 0) \rightarrow \left(\frac{1}{n-1}, \frac{1}{n-1}, \dots, 0, \dots, \frac{1}{n-1} \right)$$

was wir formal durch den Übergang $H \rightarrow M$ kennzeichnen wollen. Die Seite ohne Links wird also ersetzt durch eine Seite, die auf **alle** Seiten im Internet mit gleicher Wahrscheinlichkeit verlinkt. Das entspricht dem Verhalten des Zufallssurfers für diese Seite. Wenn die j -te Zeile in H eine solche Nullzeile ist und ersetzt wird, dann wird das j -te Element in der ersetzten Zeile zu Null gesetzt, um Selbstreferenzierung auszuschließen.

Um das Problem der geschlossenen Zyklen in der Hyperlinkmatrix zu lösen, wird die Übergangsmatrix noch einmal modifiziert: $M \rightarrow G$ mit

$$G = \alpha M + (1 - \alpha) \frac{1}{n} E$$

wobei $\alpha \in [0, 1]$ und E ist die $(n \times n)$ -Einheitsmatrix. In der Matrix G wird die **tatsächliche** Linkstruktur des Internets, gewichtet mit einem Faktor α , überlagert mit einer **gleichverteilten** Linkstruktur auf alle Seiten im Internet, die mit dem Faktor $(1 - \alpha)$ gewichtet wird. Man nennt G die **Google-Matrix**.

Der Pagerank-Vektor ist wieder die Lösung des Eigenwert-Problems

$$G^t \cdot r = r$$

d.h. r ist der Eigenvektor der transponierten Googlematrix zum Eigenwert 1. Die Matrix G ist **zeilen-stochastisch**, weil die Summe aller Zeilenelemente immer 1 ergibt. Eine solche Matrix hat **immer** einen Eigenwert 1, dies ist der maximale Eigenwert und der zugehörige Eigenvektor hat nur positive Elemente (Satz von Perron-Frobenius). D.h. der Pagerank-Vektor ist immer definiert.

Zur Lösung des Eigenwertes kann man in Matlab den Befehl `eig` verwenden. Dieser ist aber für sehr große Matrizen (wie es die Google-Matrix ist) nicht effizient, da er **alle** Eigenvektoren und Eigenwerte der Matrix berechnet. Wir sind aber nur an **einem** Eigenvektor interessiert: dem zum größten Eigenwert!

Die Theorie besagt, dass wenn man einen Vektor wiederholt (iterativ) mit einer Matrix multipliziert, dann richtet er sich nach dem Eigenvektor aus, der zum größten Eigenwert der Matrix gehört (die sogenannte **Potenzmethode**). D.h. der Vektor $r^{(k)}$ in der Iteration

$$r^{(k+1)} = G^t \cdot r^{(k)}$$

konvergiert gegen den gesuchten Pagerank-Vektor

$$r^{(k)} \rightarrow r$$

Die Potenzmethode zur Berechnung des Pageranks ist sehr viel effizienter, als die Berechnung mit anderen Methoden aus der numerischen linearen Algebra.